

# EDS Lab Assignments

## Practical2

**Name:** Vedant Deshmukh

**PRN:**202501040046

### 2.1.1 List Operations

#### Problem Statement:

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu.

Depending on the choice selected, the program should perform the following actions:

- **Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit:** Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

#### Code:

```
l1 = []
while True:
    print("1. Add")
    print("2. Remove")
    print("3. Display")
    print("4. Quit")

    n=int(input("Enter choice: "))
    if n==1:
        add = int(input("Integer: "))
        l1.append(add)
```

```

        print(f"List after adding: {l1}")
    elif n==2:
        if len(l1)==0:
            print("List is empty")
        elif len(l1)!=0:
            remove=int(input("Integer: "))
            if remove not in l1:
                print("Element not found")
            else:
                l1.remove(remove)
            print(f"List after removing: {l1}")
    elif n==3:
        if len(l1)==0:
            print("List is empty")
        else:
            print(l1)
    elif n==4:
        break
    else:
        print("Invalid choice")

```

## Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '2.1.1. List operations' is visible, detailing the requirements for a menu-driven program to manage a list of integers. The main editor shows the Python code for 'listOps.py', which implements the logic described in the problem statement. The code includes a while loop for a continuous menu, with options to add, remove, display, or quit the list. Test results are shown on the right, indicating that 2 out of 2 shown test cases and 1 out of 1 hidden test cases passed. A detailed view of 'Test case 1' shows the expected and actual outputs, confirming the program's behavior for the provided inputs.

## 2.1.2 Dictionary Operations

**Problem Statement:**

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

**Operations to be Performed:**

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

**Input and Output Format:**

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

**Code:**

# Initial dictionary with 10 predefined records

```
student = {  
    1: "Amit",  
    2: "Riya",  
    3: "Kiran",  
    4: "Neha",  
    5: "Arjun",  
    6: "Pooja",  
    7: "Rahul",  
    8: "Sneha",  
    9: "Vikram",  
    10: "Anjali"  
}
```

```
# write your code here..  
print("Original Dictionary:", student)
```

```
new_key = int(input())  
new_value = input()  
student[new_key] = new_value
```

```

print("After Insertion:", student)

update_key = int(input())
new_val = input()
if update_key in student:
    student[update_key] = new_val
print("After Update:", student)

del_key = int(input())
if del_key in student:
    del student[del_key]
print("After Deletion:", student)

print("Traversing Dictionary:")
for key, value in student.items():
    print(f"{key} : {value}")

```

## Output:

The screenshot displays a C++ IDE interface with a problem statement on the left and a code editor on the right. The problem statement, titled "2.1.2. Dictionary Operations", describes a program to perform insertion, update, deletion, and traversal on a dictionary. It includes a list of operations to be performed, input/output format, and sample test cases. The code editor shows a C++ program that implements these operations using a dictionary. The program starts with a predefined dictionary of 10 records. It then performs the following steps: 1. Insertion: Reads a key and a value, inserts them into the dictionary, and displays the dictionary. 2. Update: Reads a key and a new value, updates the value of the key in the dictionary, and displays the dictionary. 3. Deletion: Reads a key, deletes it from the dictionary, and displays the dictionary. 4. Traversal: Traverses the dictionary and displays all key-value pairs. The test results at the bottom show that the program passed all test cases, with an average time of 0.007 s and a maximum time of 0.009 s.

**Problem Statement:**

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

**Operations to be Performed:**

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

**Input and Output Format:**

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

**Note:**

- All operations must be performed using dictionary methods.
- Perform deletion only if possible, leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/outputs.

**Sample Test Cases**

**Code Editor:**

```

1 # Initial dictionary with 10 predefined records
2 student = {
3     1: "Amit",
4     2: "Riya",
5     3: "Kiran",
6     4: "Neha",
7     5: "Arjun",
8     6: "Pooja",
9     7: "Rahul",
10    8: "Sneha",
11    9: "Vikram",
12    10: "Anjali"
13 }
14 print("Original Dictionary:", student)
15 key1 = int(input())
16 value1 = input()
17 student.update({key1: value1})
18 print("After Insertion:", student)
19 key2 = int(input())
20 value2 = input()

```

**Test Results:**

2 out of 2 shown test case(s) passed  
2 out of 2 hidden test case(s) passed

**Test case 1:**

Expected output: Original Dictionary: (1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali')

Actual output: Original Dictionary: (1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali')

After Insertion: (1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali')

## 2.2.1 Linear Search Technique

### Problem Statement:

Write a program to check whether the given element is present or not in the array of elements using linear search.

### Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

### Output format:

- If the element is found, print the index.
- If the element is not found, print **Not found**.

### Code:

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i  
    return -1
```

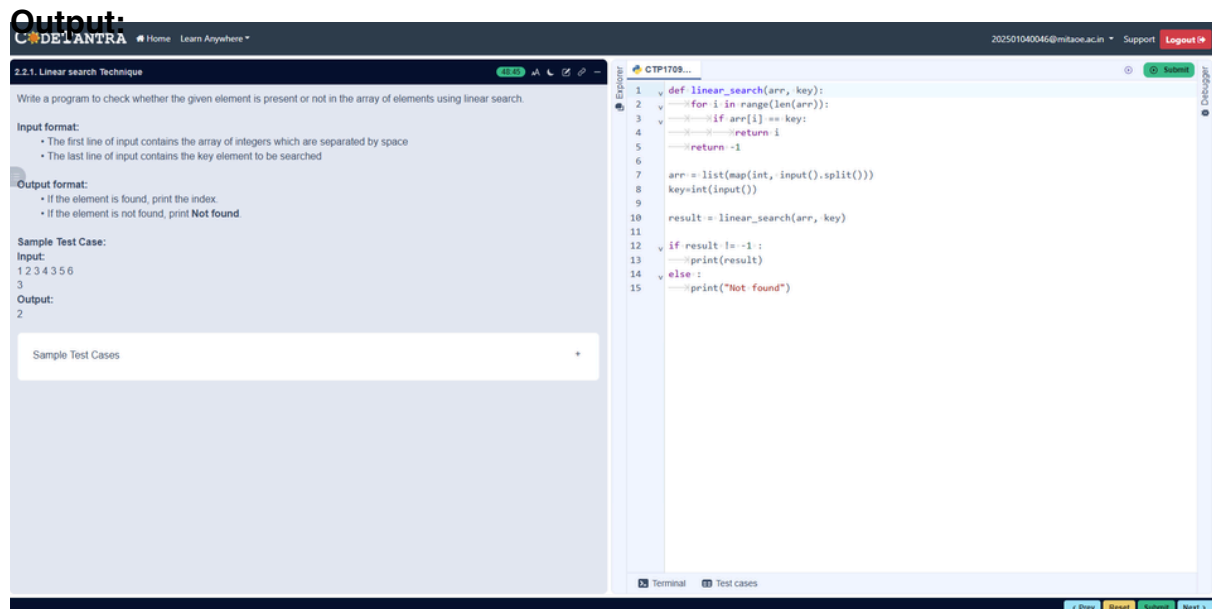
```
arr = list(map(int, input().split()))  
key = int(input())  
  
result = linear_search(arr, key)
```

if result != -1:

    print(result)

else:

    print("Not found")



## 2.2.2 Captain of the Team

### Problem Statement:

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

## Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

## Output Format

The output should be the height (in centimeters) of the tallest player.

## Code:

```
heights = list(map(int,input().split()))
tallest_player = max(heights)
print(tallest_player)
```

## Output:

The screenshot displays a coding platform interface for a problem titled "2.2. Captain of the Team". The problem description states: "You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team." The input format specifies that the first line of input contains 11 integers representing player heights, separated by spaces. The output format requires the height of the tallest player. A "Sample Test Cases" section is present but empty.

The code editor on the right shows a Python solution:

```
1 '''h=list(map(int,input().split()))
2 max=h[0]
3 for x in h:
4     if x>max:
5         max=x
6 print(max)
7
8 h=list(map(int,input().split()))
9 max=max(h)
10 print(max)
```

Below the code editor, the execution results are shown:

- Average time: 0.003 s (2.87 ms)
- Maximum time: 0.004 s (4.00 ms)
- 1 out of 1 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test case 1 details:

| Expected output                             | Actual output                               |
|---------------------------------------------|---------------------------------------------|
| 171 169 185 156 174 191 186 190 187 172 160 | 171 169 185 156 174 191 186 190 187 172 160 |
| 191                                         | 191                                         |

At the bottom, there are buttons for "Terminal", "Test cases", "Prev", "Reset", "Submit", and "Next".